

Evaluation and Final Project of INFO-H-414

Swarm Intelligence

Second Session

6 July, 2026

1 Modality

The evaluation of the course consists of three parts. To pass the course, you must pass each of the three parts separately with a minimum mark of 10/20. If you pass all three parts, your final mark will be computed as the average of the marks you obtained in each of them.

- **Project.** You will need to solve a swarm robotics challenge and submit a report with your results. The details of the project are provided in Section 2 of this document.
- **Swarm robotics: oral exam.** You will answer questions from Prof. Birattari on the contents of the course concerning swarm robotics. During this part, you might also be asked to answer some questions about your project, should there be any doubts about your methodology or the results you obtained.
- **Optimisation: oral exam.** You will answer questions from Prof. Dorigo on the contents of the course concerning optimisation.

IMPORTANT: if you already passed any of the three parts during the first session, you will not need to pass them again.

1.1 Timeline

- The project submission deadline is **14 August** at 23:59.
- Delay on the submission will entail a penalty of 1 point every 12 hours of delay on the final evaluation of the project. Maximum delay is **16 August**, at 23:59. After this deadline, you fail the exam.

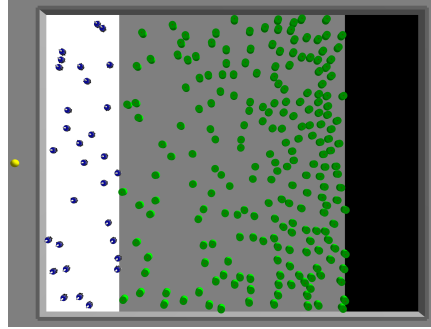


Figure 1: A top view of the arena.

2 Project: *tunnelling*

In this project, we consider a tunnelling task. In a tunnelling task, the swarm must clear a path through an environment that is filled with obstacles in order to reach a target location. They must do so while also trying to avoid contaminating the target. This requires careful cooperation between the robots, since they should all reach the target, but they must keep the obstacles out of it. Additionally, the robots are not aware of the exact location of the target, so they must rely on ambient cues to navigate the environment.

You are asked to produce and assess control software for a group of tunnelling robots. The goal is to maximise the performance of the swarm by designing a cooperative collective behaviour.

2.1 Problem definition

The robot swarm operates in a rectangular arena with grey floor, as shown in Figure 1. On the right-hand side of the arena there is a black rectangle that indicates the location of the target. The robots are initially placed on a white rectangle on the left-hand side of the arena. The area between the starting location of the robots and the target is filled with cylinders (the obstacles) that can be transported by the robots. These cylinders also have red LEDs on top. Finally, there is an ambient light source on the left-hand side of the arena.

Goal The goal of the robot swarm is to reach the target location, without contaminating it in the process. Each experimental run lasts 1,000 seconds — 10,000 ARGoS time-steps. You are free to reduce this time if the solution you have developed effectively addresses the problem in a shorter time. If you reduce the simulation time, please clarify and justify it in the report. The overall performance of the swarm is measured by the total number of robots that can be found in the target minus the number of obstacles in the

target at the end of the run. More precisely, it is calculated as

$$\sum_{i=1}^N R_i(T) - \sum_{j=1}^M O_j(T), \quad (1)$$

where $T = 10,000$ is the total duration of the run, N is the number of robots in the swarm, M is the number of obstacles in the arena,

$$R_i(T) = \begin{cases} 1, & \text{if robot } i \text{ is in the target at time } T \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

and, similarly,

$$O_j(T) = \begin{cases} 1, & \text{if obstacle } j \text{ is in the target at time } T \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

ARGoS is configured to automatically dump data on a file whose name can be changed in `tunnelling.argos` (see Section 2.3). This file contains a table with four columns:

- **CLOCK:** Column indicating the current step
- **ROBOTS:** Column indicating the number of robots currently in the target
- **OBSTACLES:** Column indicating the number of obstacles currently in the target
- **SCORE:** Column indicating the current score

Swarm composition Originally, the swarm consists of 10 homogeneous robots. See Table 1 for a summary of the sensors and actuators available. For this project, you will need to use a pair of actuators of the foot-bot that we have not used in the practice sessions: the gripper and the turret. The gripper can be used by a robot to attach itself to an object. The turret is used to orient the gripper around the robot. For details on how to use both the gripper and the turret, please refer to the foot-bot documentation, which you can find among the files of the Getting Started practice session.

In the file `tunnelling.argos`, we provide you with a swarm of 10 robots. You will need to choose the number of tunnelling robots that you want to design your control software for. In order to set your desired number of tunnelling robots, find the lines with the following format (usually line 71):

```
<!-- You can play with the number of tunnelling robots
by changing the 'quantity' attribute -->
```

Change the quantity value on the following line to the desired number of robots. This will place the specified number of robots of that type into the arena.

Table 1: Sensors and actuators available to the robots

Available sensors	Available actuators
<code>robot.motor_ground</code>	<code>robot.wheels</code>
<code>robot.range_and_bearing</code>	<code>robot.range_and_bearing</code>
<code>robot.colored_blob_omnidirectional_camera</code>	<code>robot.leds</code>
<code>robot.proximity</code>	<code>robot.gripper</code>
<code>robot.light</code>	<code>robot.turret</code>
<code>robot.id</code>	

2.2 Objective

The goal of this project is to design, implement and evaluate control software that aims to maximise the number of robots while minimising the number of obstacles in the target. The control software has to demonstrate a *co-operative* behaviour: one that takes advantage of the swarm's principles. However, this project is not only about optimising the performance of the swarm. Instead, we are also interested in seeing solutions that make use of the swarm robotics principles that you learned during the course.

You are also asked to investigate an important property of robot swarms: scalability. To this end, you will conduct experiments while the robots operate with the control software you designed. Since the performance is stochastic, you will need to run your control software multiple times in each setting to get a reliable estimate of the performance. Run ARGoS 10 times on different seeds, without making any modifications to your original implementation. In your report, please report the results obtained from the experiments (the performance) in the form of boxplots, where each box corresponds to the 10 repetitions. Bonus points are given if the studies are conducted with a higher resolution of swarm sizes.

Additionally, create and analyse a plot that shows the trend over time of the number of robots in the target, and the variance across the 10 runs. Create also a similar plot, but with the number of obstacles in the target instead. For this part, you do not need to analyse all the swarm sizes that you experiment with. Instead, select a few representative sizes that reflect how these trends evolve with the size of the swarm and plot them together¹.

Try to answer the following questions when analysing the results: How does the performance scale with the number of robots? Is the performance always increasing? Are there certain sizes for which the performance of the robots varies more notably than in others? What are the possible reasons that would explain the observed development of the performance?

Bonus points can also be earned for implementing a second simplistic

¹See, for example, <https://stackoverflow.com/questions/12957582/plot-yerr-xerr-as-shaded-region-rather-than-error-bars>

baseline behavior to which you can compare your solution. Ideally, this baseline behavior should serve as a reference point from which compare the complexity and added value of your strategy.

Remember to follow the principles of swarm robotics and apply what you learned during the course. Simple and reactive tend to work well and are very representative of the swarm robotics principles. Be aware that, for the project evaluation, the analysis is as important as the implementation. Make sure that the information provided in the report is meaningful, clearly written and complete. A good analysis is the one that discusses how design choices affect the performance of the robots—which are supported by the results.

2.3 Setting up ARGoS

- Download the experiment files: SR_Project_H414.zip
- Unpack the archive into your \$ARGOS_INSTALL_PATH directory and compile the code

```
$ unzip SR_Project_H414.zip          # Unpacking
$ cd SR_Project_H414                 # Enter the directory
$ ./build.sh                         # Compile the code
```

- Set the environment variable ARGOS_PLUGIN_PATH to the full path in which the build/ directory is located:

```
$ export ARGOS_PLUGIN_PATH=\
$ARGOS_INSTALL_PATH/SR_Project_H414/build/:$ARGOS_PLUGIN_PATH
```

You can also put this line into your \$HOME/.bashrc file, so it will be automatically executed every time you open a terminal.

- Run the experiment to check that everything is OK:

```
$ cd $HOME/SR_Project_H414           # Enter the directory
$ argos3 -c tunnelling.argos         # Run the experiment
```

If the usual ARGoS GUI appears, you're ready to go.

Switching the visualization on and off. The experiment configuration file allows you to launch ARGoS both with and without visualization. When you launch ARGoS with the visualization, you can program the robots interactively exactly like you did during the course. Launching ARGoS without the visualization allows you to run multiple repetitions of an experiment automatically, e.g., through a script. By default, the script launches ARGoS

in interactive mode. To switch the visualization off, just substitute the visualization section with: `<visualization />`, or, equivalently, comment out the entire qt-opengl section.

Loading a script at init time. When you launch ARGoS without visualization, you cannot use the GUI to set the running script. However, you can modify the XML configuration file to load automatically a script for you. At line 47 of `tunnelling.argos` you'll see that the Lua controller has an empty section `<params />`. An example of how to set the script is at line 50 of the same file. Just change line 47 to the example and set the script attribute to the file name of your script.

Changing the random seed. When you want to run multiple repetitions of an experiment, it is necessary to change the random seed every time. To change the random seed, set the value at line 11 of `tunnelling.argos`, attribute `random_seed`.

Making ARGoS run faster. Sometimes ARGoS is a little slow, especially when many robots and many sensors are being simulated. Sometimes you can make ARGoS go faster by setting the attribute `threads` at line 9 of `tunnelling.argos`. Experiment with the values, because the best setting depends on your computer.

2.4 Deliverables and evaluation

The deliverable will be a zip file containing the following items (please create a folder for each one):

- Report
- Code + README
- Results + Demonstrative video

Submission will be done via the Assignments of Microsoft TEAMS². **IMPORTANT:** Please make sure that you actually turn in your assignment³. If you only upload your files, but do not turn in the assignment, your project might not be considered for evaluation. If you are unsure if your assignment has been turned in correctly, please contact the assistants.

²Please, contact the assistants if you encounter problems with TEAMS.

³<https://support.microsoft.com/en-us/topic/e25f383a-b747-4a0b-b6d5-a2845a52092b>

Report: The final part of this project consist in writing a report of **max 7 pages**—not counting the references⁴. The report must be written in English and it must describe your work. You have to explain both the idea and the implementation of your solution. However, you should not get into the technical details of the implementation (that is what the well-documented code is for), but rather provide a high-level overview of your implementation. A graphical representation of the control software (for example, in the form of a finite-state machine) is mandatory. Additionally, you have to analyse your results, that is, discuss the limitations and strengths of your approach. To better manage your space, you should keep the description of the problem as short as possible.

The report must be typeset using the template of Lecture Notes in Computer Sciences⁵ (LNCS – Springer), available in the TEAMS assignment in the file: `Templates_report.zip` The template offers a specific page layout, **do not** use any software that changes the page layout (for example the L^AT_EX package `geometry`).

The format of your report will be that of a scientific article, including abstract, introduction, short description of the problem, description of the solution, results, conclusions and references (a few examples on how to structure your project document are included in the file: `Example_scientific_articles.zip`, also attached to the assignment).

Code: You have to submit your code following the following guidelines:

- The Lua script that you developed, commented and well-structured.
- A README file explaining how to use your code.
- The code should be ready to be tested by us. This means the code should not generate any errors when executed. A common problem is hard-coding file paths that depend on your system. If you need to include these paths, expose them as a global variable and explain in the README what needs to be changed before execution.
- You must implement your own code. **Plagiarism will be strongly penalised.**

Results:

- Create a *csv* file with the results generated by ARGoS for each experiment. Also, include a copy of all your box plots and statistical tests carried out to analyse the performance of your implementations.

⁴You may add as many pages of bibliographical references as you need to support your work.

⁵<https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines>

- Include a video of your control software with the swarm composition that you chose to design your software for at first. You can create a series of images in ARGoS by pressing the round record button before starting your experiment. The generated images can be transformed into a video by the following command:

```
ffmpeg -r 10 -i frame_%010d.png video.mp4.
```

You can also decide to record your screen using a screen capture program. **Do not** record your screen using an external camera (webcam, phone). The resulting video will be several minutes long. Please speed up the video by a proper factor so that the resulting video is at most one minute long. Using the following command, you will speed up the video by 10x:

```
ffmpeg -i video.mp4 -r 60 -filter:v "setpts=0.1*PTS" video_10x.mp4
```

3 Contacts

Marco Dorigo	for general questions
Mauro Birattari	for general questions
Guillermo Legarda Herranz	for swarm robotics
Jeanne Szpirer	for swarm robotics